

ITIL · PROBLEM MANAGEMENT

# Problem Management Techniques

Structured Exam Study Guide & Scenario Practice

6 techniques compared

Focus indicators

Diagrams & flows

Multi-answer practice

**FIND · ANALYSE · FIX · PREVENT**

Prepared 2026 · Based on your Problem Management notes

# How to Use This Guide

Your exam is **scenario-based** and **more than one option can be correct**. The dominant skill in this module is **picking the right technique for the situation** — so each scenario keyword (facts, chain of failures, component impact, past outages, after-the-fix) maps to one technique. Treat every option as its own true/false statement.

READ THE TRIGGER

"prove with facts" → KT · "chain of failures / AND-OR" → FTA · "if X fails, impact?" → CFIA · "past outages / MTTR" → SOA.

JUDGE EACH OPTION

Several statements can be true together — don't stop at the first correct one.

WATCH THE TRAPS

Each technique has an "Exam Trap" box showing the exact distinction examiners test.

## Focus Indicator — what the signs mean

- HIGH FOCUS

Very likely in scenarios · master fully
- MEDIUM FOCUS

Common distinctions · know the differences
- SUPPORTING

Context & recall · lighter revision

## Focus Priority Map

| Topic                                         | Focus    | Why it matters for scenarios                                      |
|-----------------------------------------------|----------|-------------------------------------------------------------------|
| Technique Selection & Comparison              | ●●● HIGH | The #1 question type — match the scenario to the right technique. |
| Incident vs Problem                           | ●●● HIGH | Symptom vs cause; the foundation of every question.               |
| Kepner-Tregoe (KT) — Is/Is Not                | ●●● HIGH | Facts over assumptions; step order; What/Where/When/Extent.       |
| FTA vs CFIA                                   | ●●● HIGH | Cause (work backward) vs Impact (component forward).              |
| SOA — Reliability/Maintainability/MTTR        | ●●● HIGH | Learn from past outages; Pareto 80/20.                            |
| Mitigation vs Contingency                     | ●●○ MED  | Prevent (before) vs Recover (after).                              |
| Prioritization (Impact+Probability+Effort)    | ●●○ MED  | Rate 1–5; consistent framework.                                   |
| Cause-Effect / Fishbone (5 Whys, Multivoting) | ●●○ MED  | Organise causes across Four Dimensions.                           |
| CFIA failover codes (S/A/H/W/C)               | ●●○ MED  | S = single point of failure = highest risk.                       |
| Four Dimensions                               | ●●○ MED  | Don't assume technology is always the cause.                      |
| Brainstorming · Problem Review                | ●●○ MED  | No criticism; no blame; learning & buy-in.                        |
| Detailed step sequences & examples            | ●○○ SUP  | Recognise the flow; light memorisation.                           |

# 1 · Foundations — Problem vs Incident

... HIGH

## INCIDENT

Something is **broken** — the **symptom**. "What happened?"

## PROBLEM

The **underlying cause** of one or more incidents — the **cause**. "Why did it happen?"

## THE CLASSIC EXAMPLE

800 users can't access shared drives → **800 Incidents** (users report it) but **1 Problem** (storage server failure = root cause). Many incidents can share one problem.

## Problem Management — definition & objectives

### DEFINITION

The practice of **identifying, analysing, managing, and preventing** problems to reduce future incidents. Goal is **not just fixing today's issue** but **stopping it recurring**.

- ✓ Find the **root cause**
- ✓ Reduce the **number** of incidents
- ✓ Reduce the **impact** of incidents
- ✓ Prevent future problems
- ✓ Improve continuously using lessons learned

### MEMORY TRICK

Incident → "What happened?" · Problem → "Why did it happen?" · Problem Management → "Stop it happening again."

### EXAM TRAP

Don't confuse **Problem Management** (find & fix the cause) with a **Problem Review** (held *after* the fix to learn). Fix first, review second.

## 2 · Four Dimensions, Mistakes & Communication

●●○ MEDIUM

### FOUR DIMENSIONS — WHERE TO LOOK FOR CAUSES

Check all four so you don't tunnel on technology.

| Dimension                 | Think about                     |
|---------------------------|---------------------------------|
| Organisations & People    | Skills, communication, staffing |
| Information & Technology  | Software, hardware, data        |
| Partners & Suppliers      | Vendors, third parties          |
| Value Streams & Processes | Procedures and workflows        |

These dimensions are reused everywhere: Fishbone "main bones", FTA cause search, SOA cause analysis, brainstorming prompts.

### Two mistakes to avoid

#### JUMPING TO SOLUTIONS

Fixing before understanding the real cause.

#### ANALYSIS PARALYSIS

Over-analysing and delaying action.

#### ITIL PRINCIPLE

Do **just enough analysis** to take **timely action** — the balance between the two mistakes above.

### Communication — stakeholders must know

What happened?

What caused it?

What's being done?

Who's responsible?

How prevented next time?

#### EXAM TIP

"Don't assume the technology is always the cause." Good communication **builds confidence**.

### 3 · Prioritisation & Risk Countermeasures

●●○ MEDIUM

#### WHY "A CONSISTENT SET"?

Many techniques exist from different authors — they differ in categories, priorities, and steps. ITIL makes them **consistent** so they work together.

#### Common Prioritisation Framework — rate each 1–5

| Factor      | Meaning                        |
|-------------|--------------------------------|
| Impact      | How serious is the effect?     |
| Probability | How likely is it to happen?    |
| Effort      | Time, cost & resources needed? |

#### MEMORY TRICK

**Impact + Probability + Effort = Priority.** Highest priority = **High Impact + High Probability + Low/Acceptable Effort.**

#### Risk Countermeasures — two ways to handle risk

##### MITIGATION — PREVENT (BEFORE)

Stop the problem before it happens.

Fireproof materials · regular server maintenance · redundant power

##### CONTINGENCY — RECOVER (AFTER)

Reduce impact after it happens.

Fire extinguisher · backup/failover server

#### MEMORY TRICK

Mitigation → **Minimise** chance (Before). Contingency → **Cope** after (After).

#### EXAM TRAP

A **backup server** = Contingency (recover). **Maintenance** that stops the failure = Mitigation (prevent). Classify by *when* it acts — before vs after the failure.

## 4 · Technique Selection ★ (Master Comparison)

... HIGH — #1 EXAM TOPIC

### THE DECISION THAT WINS MARKS

Every scenario points at one technique. Find the **trigger phrase** → pick the technique. This is the most common scenario question in the module.

| Technique                 | Main question it answers                                 | Scenario trigger                                        |
|---------------------------|----------------------------------------------------------|---------------------------------------------------------|
| Brainstorming             | What ideas do we have?                                   | Stuck, need many ideas, team input, buy-in              |
| Cause-Effect (Fishbone)   | Why did it happen? (all possible causes)                 | Complex problem, organise many causes visually          |
| Kepner-Tregoe (KT)        | Which cause is <b>proven by facts</b> ?                  | Need facts not guesses; "Is / Is Not"; verify cause     |
| Fault Tree Analysis (FTA) | What <b>chain of failures</b> caused the outage?         | AND/OR logic, dependencies, work backward from failure  |
| CFIA                      | If a <b>component</b> fails, what is the <b>impact</b> ? | Single points of failure, failover, redundancy planning |
| SOA                       | What can we learn from <b>past outages</b> ?             | Recurring outages, MTTR, reliability/maintainability    |
| Problem Review            | What do we learn <b>after the fix</b> ?                  | Post-resolution, no blame, lessons learned              |

### ONE-LINE MEMORY AID

Brainstorm = **ideas** · Fishbone = **find causes** · KT = **prove the cause** · FTA = **trace the chain** · CFIA = **component impact** · SOA = **learn from outages** · Review = **after the fix**.

### EXAM TRAP — THE THREE FAVOURITE CONFUSIONS

- **FTA = Cause** (why did it fail, work backward) vs **CFIA = Impact** (what if this component fails, look forward).
- **SOA = Lessons** from *past* outages (reduce MTTR) — not live troubleshooting.
- **KT** proves the cause with evidence and **never jumps to solutions**; Fishbone/Brainstorming *generate* candidate causes.

### Timeline view — before / during / after

**BEFORE** (proactive): Brainstorm · FTA · CFIA → mitigations & contingencies

**DURING**: Fishbone · KT · FTA → find & prove root cause

**AFTER** outage: SOA (learn) | **AFTER** full fix: Problem Review (lessons)

## 5 · Brainstorming

●●○ MEDIUM

### DEFINITION

A team technique where everyone shares ideas **one by one (round-robin)** until no more ideas — **no criticism or judging** during idea generation. Goal: **many ideas quickly** (quantity first, quality later).

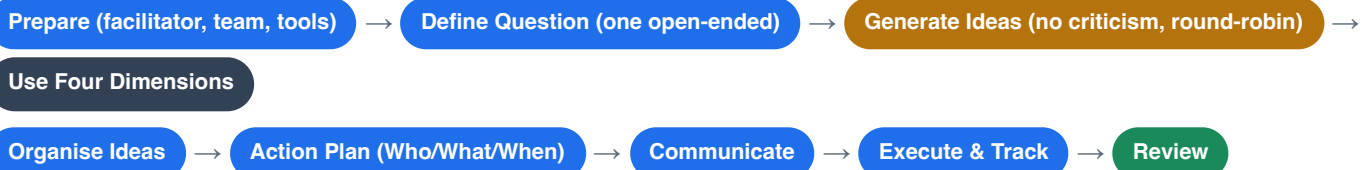
### KEY RULES

Everyone participates · all ideas welcome · no criticism while generating · build on others' ideas · quantity first.

### WHEN TO USE — ANYTIME

**Before** → prevent (mitigations/contingencies) · **During** → causes/workarounds/solutions · **After** → problem review · **When stuck** → fresh ideas.

### The 9 steps — PQIOACER



### FACILITATOR'S ROLE

Record ideas · keep on track · ensure everyone participates · **prevent criticism**.

### MEMORY TRICK

"Think first, judge later." More ideas → better solutions. Buy-in: people support ideas they helped create.

### EXAM TRAP

The **#1 rule is no criticism during generation**. Any option that says "evaluate/reject ideas as they come" is wrong — judging happens later, in the Organise step.

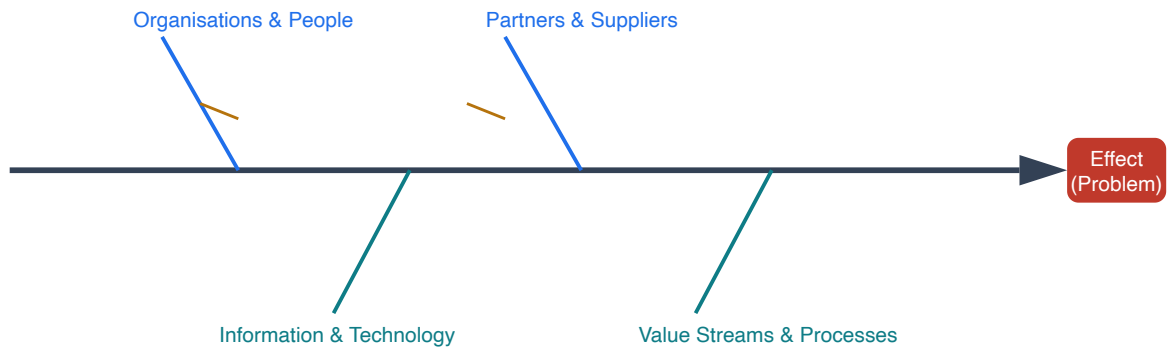
## 6 · Cause-Effect Analysis (Fishbone / Ishikawa)

●●○ MEDIUM

### DEFINITION

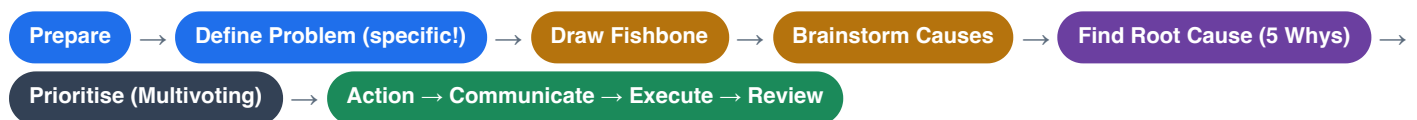
Links **causes** to their **effect** to find the **root cause**, separating the real cause from symptoms. Goal: fix the cause, not the symptom.

### Fishbone diagram — Four Dimensions as the main bones



Trunk = problem · Main bones = Four Dimensions · Branches = primary causes · Small branches = secondary causes.

### Steps



| Term        | Meaning                                                    |
|-------------|------------------------------------------------------------|
| Effect      | The problem being investigated                             |
| 5 Whys      | Keep asking "Why?" until the root cause                    |
| Multivoting | Team votes to prioritise which causes to investigate first |

### GOOD VS BAD PROBLEM STATEMENT

✓ "The system crashes 3 times a day during peak hours." ✗ "The system is unreliable." A good statement gives What, Who, How much, and Evidence.

### EXAM TRAP

Fishbone **organises possible causes** and uses **5 Whys** to reach the root, and **Multivoting** to prioritise. It shows candidate causes — it does not *prove* them with evidence (that's KT).

## 7 · Kepner-Tregoe (KT) Root Cause Analysis

... HIGH

### DEFINITION

A **structured RCA** that finds the real cause using **facts, not assumptions**. Its signature move: compare **what IS happening vs what IS NOT** — the differences reveal the cause.

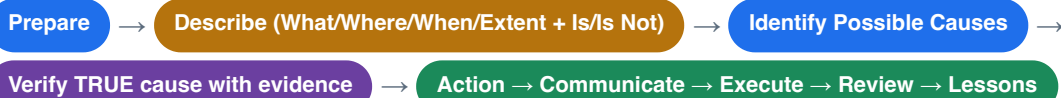
### Describe the problem precisely — 4 questions + Is/Is Not

| Question | Meaning                                |
|----------|----------------------------------------|
| What?    | What is the problem?                   |
| Where?   | Where does it happen?                  |
| When?    | When / how often?                      |
| Extent?  | Who/what is affected, and by how much? |

### "IS / IS NOT" COMPARISON (THE KT TRADEMARK)

e.g. Poor performance **IS** on Legato team sites but **IS NOT** on business-facing sites → narrows the investigation fast.

### Steps — one step must finish before the next



### VERIFY THE TRUE CAUSE

Choose the cause that **explains all symptoms**, **matches the Is/Is-Not facts**, and needs the **fewest assumptions**. Missing evidence? Gather more.

### MEMORY TRICK

**KT = Know the Truth** · Facts ✓ Assumptions ✗ · **Describe** → **Compare** → **Verify** → **Act**. Never jump to the solution.

### EXAM TRAP — THE KEY KT CONCEPT

KT differs from brainstorming because it **does not jump to solutions**. Order: Describe precisely → Compare Is/Is-Not → Verify with evidence → *only then* choose the fix. **KT = Facts** → **Evidence** → **Root Cause** → **Solution**.

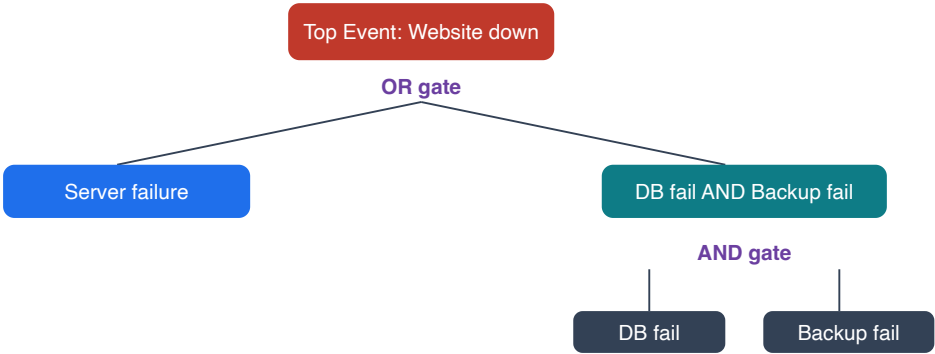
# 8 · Fault Tree Analysis (FTA)

... HIGH

## DEFINITION

Traces the **chain of events** that led to a failure, using a tree of logic gates. Start at the **Top Event (problem)** and work **backward** to root causes. Goal: understand how small failures combine into a major one.

## Fault-tree with logic gates



| Gate    | Meaning                                                                       |
|---------|-------------------------------------------------------------------------------|
| AND     | Problem occurs only if <b>all</b> causes happen (e.g. server AND backup fail) |
| OR      | Problem occurs if <b>any one</b> cause happens (network OR power)             |
| XOR     | Exclusive — only <b>one</b> cause occurs, not both                            |
| INHIBIT | Failure only if cause happens <b>AND</b> a specific condition exists          |

Terms: **Top Event** = main failure · **Intermediate Event** = leads to top · **Root/Base Event** = basic cause, no further analysis.

## MEMORY TRICK

Upside-down tree: **Top = Problem** → **Branches = intermediate causes** → **Roots = root causes**. Then Risk → Countermeasures → Action.

## EXAM TRAP

FTA **starts with the problem and works backward**, using AND/OR gates. After finding root causes it drives **mitigations (prevent)** and **contingencies (recover)**. It answers "why/how did it fail?" — not "impact if a component fails" (that's CFIA).

## 9 · Component Failure Impact Analysis (CFIA)

... HIGH

### DEFINITION

Determines **what happens if a component fails** — starts with a **component** and looks **forward** to its impact. Finds **single points of failure** and drives redundancy/failover planning.

### Steps



### Failover capability codes

| Code | Meaning                 | Note                              |
|------|-------------------------|-----------------------------------|
| S    | Single Point of Failure | ✗ No backup — <b>highest risk</b> |
| A    | Active-Active           | ✓ Always available — best         |
| H    | Hot Standby             | 🔥 Ready immediately               |
| W    | Warm Standby            | 🌞 Needs some time                 |
| C    | Cold Standby            | ❄️ Slowest                        |

A **CI (Configuration Item)** is any component a service needs: hardware, software, network, database, people, documentation, buildings. Prioritise action with the **Pareto Principle** (20% of actions → 80% of benefit).

### EXAM TRAP — FTA VS CFIA (VERY COMMON)

**FTA** starts with a **problem** and works **backward** to causes → "Why did it fail?" **CFIA** starts with a **component** and looks **forward** → "What happens if it fails?" **FTA = cause · CFIA = impact**. Remember **S = single point of failure = highest risk**.

## 10 · Service Outage Analysis (SOA)

... HIGH

### DEFINITION

Analyses **past service outages** to understand why they happened, why recovery took time, and how to prevent them. Goal: **fewer outages + faster recovery**. Analyse across the **Four Dimensions**.

### Steps

Prepare → Group Outages → Prioritise (Pareto 80/20) → Analyse Causes (4 Dimensions) →  
Analyse Downtime (detect/diagnose/resolve) → Improve → Communicate → Execute → Review → Lessons

### Three terms examiners love

| Term            | Meaning                            | Goal                         |
|-----------------|------------------------------------|------------------------------|
| Reliability     | How often the service fails        | Fail <b>less often</b>       |
| Maintainability | How quickly it is restored         | Recover <b>faster</b>        |
| MTTR            | Mean Time To Repair/Restore        | Reduce it (find bottlenecks) |
| Pareto 80/20    | 20% of outages cause 80% of impact | Fix the vital few first      |

### MEMORY TRICK

**SOA = Study Old Outages**. Reliability → fewer failures · Maintainability → faster recovery · SOA improves both.

### EXAM TRAP

SOA analyses **past** outages for **lessons** — not live troubleshooting. Compare: **FTA = cause** · **CFIA = impact** · **SOA = lessons learned**. Most recovery delay is usually in the **diagnosis** phase.

## 11 · Problem Review

●●○ MEDIUM

### DEFINITION

A meeting held **after a problem is resolved** to learn from it. Goal: prevent similar problems and improve future response. **No blame — only learning.**

### IT REVIEWS BOTH SIDES

Not only failures: it also asks **what went well?** as well as **what didn't** and **how to improve next time?**

### Steps

Prepare → Agree on Facts (root cause, timeline) → Review Good & Bad → Document Lessons →  
Action (Impact·Probability·Effort) → Communicate → Execute → Review → Share

### THREE OBJECTIVES OF A PROBLEM REVIEW

1) Understand **why** it happened. 2) Improve **how** similar problems are handled. 3) **Agree actions** and communicate them.

### TWO GOALS

Improve **Reliability** (prevent recurrence) *and* **Maintainability** (handle it better/faster next time). Fix → Review → Learn → Improve.

### EXAM TRAP

Problem Review is **after the fix**, **fact-based**, and **blame-free**. Don't confuse it with Problem Management (find & fix the cause) or SOA (analyse past outages, often before the underlying problem is fully resolved).

# Master Comparison & Revision

| Technique      | Main question                     | Remember                                          |
|----------------|-----------------------------------|---------------------------------------------------|
| Brainstorming  | What ideas do we have?            | No criticism; round-robin; buy-in                 |
| Fishbone       | Why? (all possible causes)        | 4 Dimensions + 5 Whys + Multivoting               |
| Kepner-Tregoe  | Which cause is proven by facts?   | Is/Is-Not; What-Where-When-Extent; no jumping     |
| FTA            | What chain of failures caused it? | Top Event → backward; AND/OR gates = <b>cause</b> |
| CFIA           | If a component fails, impact?     | CIs; S/A/H/W/C; S = highest risk = <b>impact</b>  |
| SOA            | Learn from past outages?          | Pareto 80/20; Reliability/Maintainability/MTTR    |
| Problem Review | Learn after the fix?              | After resolution; no blame; good & bad            |

## CAUSE VS IMPACT VS LESSONS

FTA = Cause · CFIA = Impact · SOA = Lessons.

## PREVENT VS RECOVER

Mitigation = Prevent (before) · Contingency = Recover (after).

## SYMPTOM VS CAUSE

Incident = Symptom · Problem = Cause.

## PRIORITISE

Impact + Probability + Effort (each 1–5).

## When in the lifecycle

BEFORE (proactive) — Brainstorming · FTA · CFIA → mitigations & contingencies



DURING — Fishbone · KT · FTA → find & prove the root cause



AFTER an outage — SOA → learn, reduce MTTR



AFTER the fix — Problem Review → lessons & continuous improvement

## GOLDEN RULE

All techniques serve one aim: **find the true root cause, reduce incidents & impact, and prevent recurrence** — using **just enough analysis to act in time**, across the **Four Dimensions**, and always **communicating & capturing lessons**.

# Scenario Practice — Part 1

More than one statement can be correct. Decide each option independently, then check the answer.

**Q1. 800 users report they cannot open the shared drive. The service desk logs each report. An engineer suspects a storage server has failed. Which statements are correct?**

- A. There are 800 incidents but potentially a single problem.
- B. The storage server failure would be the **problem** (root cause).
- C. Each user report is a separate problem.
- D. Problem Management aims to stop this recurring, not just restore access today.

**Correct: A, B, D.** Incident = symptom (many); Problem = cause (one). C confuses incidents with problems.

**Q2. A performance issue affects Legato team sites but not business-facing sites. The team wants to avoid guessing and prove the cause with evidence before fixing. Which apply?**

- A. **Kepner-Tregoe** is the best fit (facts, not assumptions).
- B. The "IS Legato / IS NOT business-facing" comparison helps narrow the cause.
- C. They should implement the first suspected fix immediately to save time.
- D. Describing What/Where/When/Extent precedes choosing a solution.

**Correct: A, B, D.** C is the trap — KT never jumps to solutions; verify with evidence first.

**Q3. During design, the team wants to know the effect of losing the primary database server and whether redundancy is justified. Which are correct?**

- A. **CFIA** is appropriate — it assesses impact if a component fails.
- B. The database marked **S** (single point of failure) is the highest risk.
- C. FTA is better because CFIA cannot consider components.
- D. Adding a hot standby (H) or active-active (A) reduces the risk.

**Correct: A, B, D.** C is wrong — CFIA is exactly the component-impact tool; FTA works backward from a failure to causes.

**Q4. A website outage is analysed as: it fails if the server fails OR (the database fails AND the backup fails). Which are correct?**

- A. This is a **Fault Tree Analysis**, starting from the Top Event and working backward.
- B. The OR gate means any single listed branch can cause the outage.
- C. The AND gate means both DB and backup must fail together.
- D. Adding a redundant backup weakens the AND branch (a mitigation).

**Correct: A, B, C, D — all four.** FTA with OR/AND logic; redundancy is a preventive mitigation.

## Scenario Practice — Part 2

**Q5. Over the past year the collaboration service had many outages. Leadership wants to reduce how often it fails and how long recovery takes. Which are correct?**

- A. **Service Outage Analysis** is the right technique.
- B. Use the **Pareto 80/20** rule to focus on the most impactful outages.
- C. Reducing outage frequency improves **reliability**; reducing recovery time improves **maintainability** (lower MTTR).
- D. SOA is used for live, in-progress incident troubleshooting.

**Correct: A, B, C.** D is wrong — SOA studies *past* outages for lessons, not live troubleshooting.

**Q6. A cross-functional team meets to solve a complex problem and needs many fresh ideas quickly. Which statements are correct?**

- A. Brainstorming should run round-robin with **no criticism** during idea generation.
- B. Judging and grouping ideas happens **later**, in the Organise step.
- C. The facilitator should reject weak ideas immediately to save time.
- D. Thinking through the Four Dimensions helps when ideas dry up.

**Correct: A, B, D.** C breaks the core rule — no criticism while generating; quantity first, quality later.

**Q7. The team must prioritise several candidate problems for investigation. Which are correct about the ITIL prioritisation framework?**

- A. Prioritise using **Impact + Probability + Effort**, each rated 1–5.
- B. Highest priority = high impact + high probability + low/acceptable effort.
- C. Only impact should be considered; effort is irrelevant.
- D. This consistent framework lets different techniques work together.

**Correct: A, B, D.** C is wrong — effort (time/cost/resources) is part of the framework.

**Q8. Proactively, a team wants both to stop a data-centre fire from happening and to limit damage if one occurs. Which are correct?**

- A. Fire-resistant materials and regular maintenance are **mitigations** (prevent).
- B. A fire extinguisher and a failover server are **contingencies** (recover).
- C. Mitigation acts before the event; contingency acts after.
- D. A backup server is a mitigation because it involves hardware.

**Correct: A, B, C.** D is wrong — a backup/failover server is a **contingency** (recovers after failure), regardless of being hardware.

## Scenario Practice — Part 3

**Q9. After a 4-hour outage is fully resolved, the team meets to capture what happened and improve. A junior member expects to be blamed. Which are correct?**

- A. This is a **Problem Review**, held after resolution.
- B. It is **blame-free** and fact-based; it reviews what went well **and** badly.
- C. Its goals include improving both reliability and maintainability.
- D. The team should agree on the facts/root cause before discussing improvements.

**Correct: A, B, C, D — all four.** Problem Review = after the fix, no blame, learn from successes and failures, agree facts first.

**Q10. A complex problem has many possible causes spread across people, technology, suppliers, and processes. The team wants a shared visual map before verifying anything. Which are correct?**

- A. A **Fishbone (Cause-Effect) diagram** organises causes by the Four Dimensions.
- B. **5 Whys** helps drill from a cause to the root cause.
- C. **Multivoting** can prioritise which causes to investigate first.
- D. Fishbone by itself statistically proves the true root cause.

**Correct: A, B, C.** D overstates it — Fishbone surfaces and organises candidate causes; proving with evidence is KT's job.

**Q11. A team is deciding which technique to use. Match the intent to the technique — which statements are correct?**

- A. "Trace the chain of failures with AND/OR logic" → **FTA**.
- B. "Prove which cause the evidence supports" → **KT**.
- C. "See the business impact if the DNS server fails" → **CFIA**.
- D. "Learn from last quarter's repeated outages to cut MTTR" → **SOA**.

**Correct: A, B, C, D — all four.** This is the core selection skill: cause-chain=FTA, prove=KT, component impact=CFIA, past outages=SOA.

**Q12. A manager insists on either fixing the very first suspected cause instantly, or spending three weeks fully modelling every possibility. Which statements reflect correct ITIL thinking?**

- A. Fixing the first guess risks **jumping to solutions**.
- B. Three weeks of modelling risks **analysis paralysis**.
- C. ITIL advises doing **just enough analysis to take timely action**.
- D. Ignoring non-technology dimensions is fine since causes are always technical.

**Correct: A, B, C.** D is wrong — always consider all Four Dimensions; technology is not always the cause.

### EXAM TECHNIQUE RECAP

1) Spot the **trigger phrase** → technique. 2) Evaluate **every option** independently. 3) Beware the favourite traps: **Incident vs Problem**, **FTA(cause) vs CFIA(impact) vs SOA(lessons)**, **Mitigation vs Contingency**, and **KT never jumps to solutions**.